Exam AWS Certified Developer - Associate DVA-C02 All Questions

View all questions & answers for the AWS Certified Developer - Associate DVA-C02 exam

Go to Exam

EXAM AWS CERTIFIED DEVELOPER - ASSOCIATE DVA-C02 TOPIC 1 QUESTIO...

Exam question from Amazon's AWS Certified Developer - Associate DVA-C02
Question #: 159
Topic #: 1
[All AWS Certified Developer - Associate DVA-C02 Questions]

A developer migrated a legacy application to an AWS Lambda function. The function uses a third-party service to pull data with a series of API calls at the end of each month. The function then processes the data to generate the monthly reports. The function has been working with no issues so far.

The third-party service recently issued a restriction to allow a fixed number of API calls each minute and each day. If the API calls exceed the limit for each minute or each day, then the service will produce errors. The API also provides the minute limit and daily limit in the response header. This restriction might extend the overall process to multiple days because the process is consuming more API calls than the available limit.

What is the MOST operationally efficient way to refactor the serverless application to accommodate this change?

A. Use an AWS Step Functions state machine to monitor API failures. Use the Wait state to delay calling the Lambda function.

B. Use an Amazon Simple Queue Service (Amazon SQS) queue to hold the API calls. Configure the Lambda function to poll the queue within the API threshold limits.

C. Use an Amazon CloudWatch Logs metric to count the number of API calls. Configure an Amazon CloudWatch alarm that stops the currently running instance of the Lambda function when the metric

exceeds the API threshold limits.

D. Use Amazon Kinesis Data Firehose to batch the API calls and deliver them to an Amazon S3 bucket with an event notification to invoke the Lambda function.

Show Suggested Answer

by fordiscussionstwo at *Oct. 6, 2023, 2:03 a.m.*

Disclaimers:
- ExamTopics website is **not** related to, affiliated with, endorsed or authorized by Amazon.
- Trademarks, certification & product names are used for reference only and belong to Amazon.

Comments

wonder_man Highly Voted 2 years, 5 months ago

Selected Answer: A

B: I don't see how the Lamba function can be configured this way

upvoted 7 times

trieudo Most Recent 1 year, 3 months ago

Selected Answer: A

A: makes the most sense in this question: Use an AWS Step Functions state machine to monitor API failures. Use the Wait state to delay calling the Lambda function.

B: SQS queue wouldn't work because the maximum delay allowed in an SQS queue is 15 minutes. ==> 'extend the overall process to multiple days' will be violated

upvoted 1 times

trieudo 1 year, 3 months ago

refer: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-delay-queues.html?utm_source=chatgpt.com

upvoted 1 times

tsangckl 1 year, 9 months ago

This appear at 17 Jun exam

upvoted 1 times

65703c1 1 year, 10 months ago

Selected Answer: A

A is the correct answer.

upvoted 1 times

KarBiswa 2 years, 1 month ago

Selected Answer: C

I would go for option C because response value always contains the limit value and which can be retrieved into the cloudwatch and can be used to block the the lambda calls

upvoted 1 times

KarBiswa 2 years ago

Changing to A

upvoted 3 times

KillThemWithKindness 2 years, 1 month ago

Selected Answer: A

The solution that will meet the requirements is to use an AWS Step Functions state machine to monitor API failures. Use the Wait state to delay calling the Lambda function. This way, the developer can refactor the serverless application to accommodate the change in a way that is automated and scalable. The developer can use Step Functions to orchestrate the Lambda function and handle any errors or retries. The developer can also use the Wait state to pause the execution for a specified duration or until a specified timestamp, which can help avoid exceeding the API limits. The other options either involve using additional services that are not necessary or appropriate for this scenario, or do not address the issue of API failures.

https://www.freecram.net/question/Amazon.DVA-C02.v2023-11-29.q68/a-developer-migrated-a-legacy-application-to-an-aws-lambda-function-the-function-uses-a-third-party-18#

upvoted 3 times

konieczny69 2 years, 2 months ago

Selected Answer: A

Who is going to orchestrate lambda invocation?
SQS is for decoupling, not for scheduled invocations.

A is the only option.

upvoted 4 times

CrescentShared 2 years, 2 months ago

Selected Answer: B

Option A with AWS Step Functions can handle the frequency of API calls by introducing a delay (Wait state) between retries after a failure due to rate limiting, it doesn't inherently solve the problem of the total number of calls per day. If the total number of necessary API calls exceeds the daily limit set by the third-party service, simply adding a delay between retries will not prevent the overall daily limit from being exceeded.

upvoted 4 times

SerialiDr 2 years, 2 months ago

Selected Answer: A

A. Use an AWS Step Functions state machine to monitor API failures. Use the Wait state to delay calling the Lambda function: This is a viable and efficient solution. AWS Step Functions can orchestrate the Lambda function invocations and manage the workflow, including handling API call rate limits. The Wait state can be used to introduce delays between API calls to ensure compliance with the rate limits. This approach also allows for handling errors and retries effectively.

B. Use an Amazon Simple Queue Service (Amazon SQS) queue to hold the API calls. Configure the Lambda function to poll the queue within the API threshold limits: While using SQS to queue API call requests is a good way to manage workload, it adds complexity to the solution. The

Lambda function would need to be modified to manage the queue and ensure API calls are made within the threshold limits. This approach might not be as straightforward and efficient as using Step Functions.

upvoted 3 times

JohnPl 2 years, 2 months ago

Selected Answer: B

B is the most operationally efficient way

upvoted 1 times

Snape 2 years, 2 months ago

Selected Answer: B

b is the answer

upvoted 1 times

rrshah83 2 years, 3 months ago

Selected Answer: B

sqs decouples lambda from api service

upvoted 1 times

chewasa 2 years, 3 months ago

Selected Answer: B

While Step Functions can be used for workflow orchestration, it may not be the most straightforward solution for handling rate limits in this scenario.

upvoted 1 times

tqiu654 2 years, 4 months ago

Selected Answer: A

ChatGPT: A

upvoted 2 times

ShawnWon 2 years, 4 months ago

B.
Option A (AWS Step Functions) might introduce unnecessary complexity and does not directly address the need to control the rate of API calls within the specified limits.

Option C (CloudWatch Logs metric and alarm) provides monitoring capabilities but doesn't offer a direct mechanism to control the rate of API calls within the Lambda function.

Option D (Kinesis Data Firehose) is designed for real-time streaming and might not be the most suitable option for this scenario, as it may not provide the fine-grained control needed to stay within the API call limits.

upvoted 3 times

dilleman 2 years, 5 months ago

Selected Answer: A

A is Correct. AWS Step Functions can be used to create a workflow to handle the API calls. You can make the Lambda function inspect the response headers from the third-party service to determine the current API call limits and then pass that to the Wait state of the state machine for proper delays.

upvoted 2 times

Digo30sp 2 years, 5 months ago

Selected Answer: B

The correct answer is (B).

Solution (B) is the most operationally efficient way to refactor the serverless application to accommodate this change. This solution allows the Lambda function to continue executing API calls even if the API call limit is reached. The Amazon SQS queue will act as a buffer for API calls that exceed the limit. The Lambda function can then poll the queue within the API limits.

upvoted 4 times